

Eventos de domínio do FestaLink

Este documento descreve os eventos publicados pelo backend, a topologia AMQP usada, o formato do payload e o consumidor responsável. Os eventos materializam a comunicação assíncrona entre o processo de API (que serve REST) e o processo de Worker (que consome eventos e empurra notificação em tempo real para o prestador).

Topologia AMQP

Todos os eventos trafegam por um único **exchange topic durável** no RabbitMQ, configurável via variável `EVENTOS_EXCHANGE`. O valor padrão é `festalink.eventos`. O processo de API é o único produtor; o processo de Worker é o único consumidor.

O Worker declara uma fila durável (`festalink.worker`, configurável em `EVENTOS_FILA_WORKER`) ligada ao exchange por uma routing key padrão `reserva.#`, configurável em `EVENTOS_ROUTING_KEYS` como lista separada por vírgula. A fila e o exchange são marcados como `durable: true` e cada mensagem é publicada com `persistent: true` , garantindo que o broker preserve os dados mesmo após reinício.

A routing key de cada evento é derivada do `type` do envelope pela conversão de PascalCase para dot-snake-case (por exemplo, `ReservaSolicitada` vira `reserva.solicitada`).

Formato base do envelope

Todo evento publicado segue o mesmo envelope:

```
{
  "type": "NomeDoEvento",
  "timestamp": "2026-05-25T23:41:45.919Z",
  "payload": { ... }
}
```

Os fields `type` e `timestamp` são fixos. O `payload` varia conforme o evento.

Eventos

Evento	Produtor (use case)	Quando dispara	Routing key	Fila do consumidor	Efeito
<code>ReservaSolicitada</code>	<code>CriarReserva</code>	Após persistir uma reserva com status <code>pendente</code>	<code>reserva.solicitada</code>	<code>festalink.worker</code>	Push para o prestador dono do salão
<code>ReservaAprovada</code>	<code>AprovarReserva</code>	Após atualizar a reserva	<code>reserva.aprovada</code>	<code>festalink.worker</code>	Push para o prestador

Evento	Produtor (use case)	Quando dispara	Routing key	Fila do consumidor	Efeito
		para aprovada			dono do salão
ReservaRecusada	RecusarReserva	Após atualizar a reserva para recusada	reserva.recusada	festalink.worker	Push para o prestador dono do salão

A publicação acontece sempre depois da transação principal, em microtarefa, fora do caminho da resposta HTTP. Uma falha de publicação é registrada no log mas não derruba a requisição que originou o evento.

Payloads

ReservaSolicitada

```
{
  "type": "ReservaSolicitada",
  "timestamp": "2026-05-25T23:41:45.919Z",
  "payload": {
    "reservaId": 4,
    "salaoId": 4,
    "clienteId": 6,
    "dataEvento": "2026-06-05",
    "horaInicio": "19:00",
    "horaFim": "22:00"
  }
}
```

ReservaAprovada

```
{
  "type": "ReservaAprovada",
  "timestamp": "2026-05-25T23:41:47.236Z",
  "payload": {
    "reservaId": 4,
    "salaoId": 4,
    "clienteId": 6
  }
}
```

ReservaRecusada

```
{
  "type": "ReservaRecusada",
  "timestamp": "2026-05-25T23:41:50.111Z",
  "payload": {
    "reservaId": 4,
    "salaoId": 4,
    "clienteId": 6
  }
}
```

Caminho pub → consume → push

1. O use case correspondente conclui sua transação na API e chama `eventPublisher.publicar(evento)`.
2. O `RabbitMQEventPublisher` serializa o envelope como JSON, deriva a routing key a partir de `evento.type` e dispara `publish` no exchange `festalink.eventos` com `persistent: true`. O channel é criado em modo `confirm`, então a publicação só resolve a Promise quando o broker confirma a entrega ao disco.
3. O RabbitMQ entrega a mensagem à fila `festalink.worker` por força do binding `reserva.#`.
4. O processo de Worker, conectado em modo consumer com `prefetch(1)`, recebe a mensagem, faz `JSON.parse` e chama `wsNotifier.notificarPrestadorDoSalao(salaoId, evento)`.
5. O notificador consulta o `SalaoRepository` para descobrir o prestador dono do salão e, se houver socket aberto, empurra o envelope serializado pelo WebSocket.
6. Em caso de sucesso, o consumer faz `ack` manual da mensagem. Em caso de erro de processamento, faz `nack` com requeue, e a mensagem volta para a fila para ser reentregue.

A propriedade central dessa topologia é a durabilidade: se o Worker estiver offline quando a API publicar um evento, a mensagem fica enfileirada em `festalink.worker` e é entregue assim que o Worker reconectar. A API segue operando independentemente do estado do Worker.

WebSocket de notificação

O `WebSocketNotifier` mora no processo de Worker e abre um servidor WebSocket na porta configurada em `WS_PORT` (padrão 3001). A autenticação acontece no handshake: o cliente conecta passando o JWT na query string (`?token=<jwt>`). O notificador valida o token, exige `papel === 'prestador'` e indexa o socket pelo `prestadorId`.

Mensagens publicadas pela API e entregues ao Worker chegam ao prestador conectado exatamente no formato do envelope acima. Ao conectar, o cliente também recebe uma mensagem informativa `{"type": "Conectado", "prestadorId": N}` para confirmar a sessão.